

Reviewer Pack — Minimal Order of Brauer Groups and Resolution Proof Width In...

Entry f2954f97f95a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 18:07:14 UTC

Minimal Order of Brauer Groups and Resolution Proof Width Inequality

Entry ID: f2954f97f95a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 18:07:14 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory (Brauer Groups) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ with n variables, the order of the Brauer group $B(\varphi)$ is at least $O(\log(n)^2/w(\varphi))$ where $w(\varphi)$ is the resolution proof width of φ .

Rationale (proposer's reasoning):

Brauer groups encode nontrivial algebraic structures that arise in the context of vector bundles and Galois theory. These groups may reveal inherent complexities in the structure of resolution proofs, providing a novel perspective on proof complexity.

Taxonomy category: algebraic_k_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 85a1f1e364e7a611

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a CNF φ with n variables, if the ratio $\log(n)^2/w(\varphi)$ from at least 80% of 30 random seeds is less than or equal to the order of the Brauer group $B(\varphi)$, then the conjecture is supported. If any seed produces a ratio greater than $B(\varphi)$ or fewer than 24 out of 30 seeds meet the support condition, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic K-theory AND brauer groups AND resolution proof complexity
- Brauer group order AND resolution proof width inequality - resolution proof width IN algebraic K-theory

Top relevant hits considered: - [http://arxiv.org/abs/1208.1599v1] Algebraic K-theory of endomorphism rings - [http://arxiv.org/abs/math/0604540v4] Enlargeability and index theory: Infinite covers - [http://arxiv.org/abs/2103.15474v4] Hermitian K-theory via oriented Gorenstein algebras - [http://arxiv.org/abs/1110.6618v2] Brauer relations in finite groups II - quasi-elementary groups of order p^{aq} - [http://arxiv.org/abs/2402.06620v2] The Brauer groups of moduli of genus three curves, abelian threefolds and plane curves - [http://arxiv.org/abs/math/0301180v6] A generalization of the topological Brauer group - [http://arxiv.org/abs/1311.1421v3] Multiplicative differential algebraic K-theory and applications - [http://arxiv.org/abs/2104.04021v2] A universal characterization of noncommutative motives and secondary algebraic K-theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for i in range(1, n+1):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def resolution_width(cnf):
        # Simplified DPLL solver to estimate resolution width
        stack = []
        while cnf:
            unit_clause = next((c for c in cnf if len(c) == 1), None)
            if not unit_clause:
                break
            literal = unit_clause[0]
            cnf.remove(unit_clause)
            for clause in cnf[:]:
                if literal in clause:
                    cnf.remove(clause)
```

```

        elif -literal in clause:
            clause.remove(-literal)
            stack.append(clause)
    return len(stack)

def brauer_group_order(n):
    # Simplified Brauer group order estimation
    return math.ceil(math.log(n, 2) ** 2)

n = random.randint(5, 40)
cnf = generate_cnf(n)
width = resolution_width(cnf)
order = brauer_group_order(n)

if width == 0:
    return {
        "metric_name": "log(n)^2 / w( $\phi$ )",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "width_is_zero"
    }

ratio = (math.log(n, 2) ** 2) / width
return {
    "metric_name": "log(n)^2 / w( $\phi$ )",
    "metric_value": ratio,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": ratio <= order,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        result = f"SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}"
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        result = f"FALSIFIED counterexample=\"width_is_zero\" first_failing_seed={first_failing_seed}"
    else:
        result = "INCONCLUSIVE mapping_undefined"

    print(result)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
zero'}  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 21, 'c  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 18, 'c  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 15, 'c  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 7, 'co  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 25, 'c  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 23, 'c  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 9, 'co  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 38, 'c  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 16, 'c  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 36, 'c  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 19, 'c  
TRIAL: {'metric_name': 'log(n)^2 / w( $\phi$ )', 'metric_value': inf, 'instances_tested': 1, 'n_max': 17, 'c  
FALSIFIED counterexample="width_is_zero" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses a simplified DPLL solver to estimate the resolution proof width, which may not accurately represent the true resolution proof width for all CNFs. This could lead to incorrect conclusions about the conjecture.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that for all tested instances, the ratio $\log(n)^2 / w(\phi)$ is infinite, suggesting that the resolution proof width $w(\phi)$ is zero | next: Investigate and refine the method for estimating the resolution proof width. Consider using more sophisticated solvers or alternative approaches to validate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 13

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19196
2	propose	ollama_remote	glm4:latest	0	0	12672
3	propose	ollama_remote	glm4:latest	0	0	13008
4	propose	ollama_remote	glm4:latest	0	0	12077
5	preregistration	ollama_remote	glm4:latest	0	0	9895
6	novelty	ollama_remote	glm4:latest	0	0	12946
7	novelty	ollama_remote	glm4:latest	0	0	9771
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12376
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9214

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7886
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	49728
12	critic	ollama_remote	glm4:latest	0	0	59010
13	judge	ollama_remote	glm4:latest	0	0	11444

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 239224 ms total latency. Provider mix: {'ollama_remote': 13}

(full prompt+response transcripts available in research/audit/f2954f97f95a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f2954f97f95a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f2954f97f95a.tar.gz (if generated)